杭州电子科技大学

计算机学院

编译原理课程实践

项目 1　预备实验

预备实验 实验指南

(课程代码：S0512080)

课程名称	编译原理课程实践
课程代码	S0512080
学时	课内 3 学时；课外完成配置与调研
文档版本	v2.5
文档日期	2026-10
编制	杭州电子科技大学 计算机学院 编译原理课程组
适用专业	计算机科学与技术、计算机科学英才班、智能计算与数据科学、计算机科学与技术（第二学士学位）

杭州电子科技大学计算机学院

2026 年 10 月

项目说明

1.1 项目目标

通过本项目，学生将：

- 认识 GCC 工具链的完整工作过程
- 熟悉 GCC、目标文件检查工具、GDB/LLDB 和 Flex 等核心开发工具
- 理解预处理、编译、汇编、链接四阶段，区分工具链阶段与编译器内部分析阶段
- 建立对国产编译器（华为方舟、毕昇等）的初步认知，培养科技报国情怀

1.2 任务索引

表 1　任务安排

任务号	任务名称	类型	建议学时	核心工具	提交要求
1.0	环境配置	验证/准备	课前完成	GCC、binutils、GDB、Flex、Git	群内打卡；报告第 1 章简要说明环境和版本
1.1	走近 GCC：从源代码到可执行文件	验证/基础	1 学时	gcc、file	hello.c、bug.c、add.c、main.c 源码 + 截图
1.2	阶段产物观察	验证/分析	1 学时	gcc -E/-S/-c、nm、objdump、ldd	四阶段命令、产物截图、-### 计划调用命令记录
1.3	GCC 编译选项实验	验证/对比	0.5 学时	-O0/-O1/-O2/-O3、-Wall、-g、-D	4 个实验的实际记录、优化对比表及结论依据；截图可汇总
1.4	Lex 工具入门	验证/工具体验	0.5 学时	Flex、gcc（noyywrap）	demo.l / numbers.l / mini_c.l 源码 + 完成数字与关键字识别规则及边界测试
1.5	国产编译器调研	调研/课外	课外完成	网络/文献检索	800 字以上调研报告 + 参考文献 ≥ 2 条

课内 3 学时用于指导与验收；环境配置在课前完成，其余调研和练习在课外补齐。所有命令在同一项目目录执行。代码块：蓝色 $ 为提示符（不输入），深蓝为命令，黑色为源代码，绿色为说明，灰色为输出；EOF 为输入结束标记。输出不需输入。

学习要求：环境自检和正常运行示例可直接执行；诊断用错误程序只用于编译检查；numbers.l、mini_c.l 仅提供练习框架。请完成指定规则，并区分给定示例、自己修改的内容与测试结果。

语言安排：本指南用 C 程序和 Flex 说明工具链。后续算法项目可自主选择实现语言；本项目的阶段产物、编译诊断与 Flex 操作仍按要求完成，不以其他语言的运行结果替代。

1.3 学习路径

建议按 1.0 → 1.1 → 1.2 → 1.3 → 1.4 → 1.5 顺序完成。

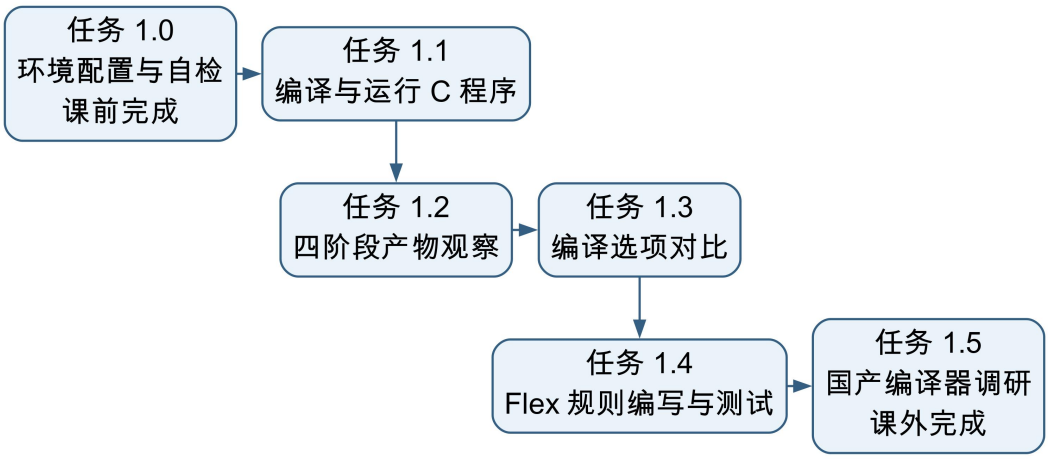


图 1　学习顺序：环境配置、编译运行、产物观察、选项比较、Flex 练习与调研

任务 1.0 环境配置

课前完成

教学目标

- 提前完成课程实践所需的全部工具安装，避免上机时挤占操作时间
- 熟悉 Linux、macOS 和 Windows 三种平台的环境差异
- 通过编译、检查、调试及 Flex 运行自检，确认工具可用。

工具清单

表 2　工具清单

工具	用途	对应任务
GCC 驱动及编译/汇编/链接工具	编译、运行 C 程序	1.1-1.3
nm/objdump 与 ldd（Linux）；otool（macOS）	观察编译产物	1.2
GDB（Linux）/ LLDB（macOS）	调试	1.3
Flex	词法分析工具	1.4
C 标准库头文件	编译 C 程序	1.1-1.4

平台安装指引

表 3　平台与工具配置

平台	安装方式
Ubuntu / Debian（首选）	sudo apt install build-essential binutils flex gdb git
macOS	安装 Xcode Command Line Tools；另安装 Flex；调试使用 LLDB
Windows（WSL 2，强烈推荐）	wsl --install -d Ubuntu-22.04
Windows（Git Bash）	仅提供部分命令环境，不能替代 WSL 的完整工具链

平台说明：macOS 的 /usr/bin/gcc 通常调用 Apple Clang，默认使用 LLDB；它不包含 GCC 的 cc1。GNU GCC 内部工具观察在 Linux/WSL 完成。本指南用 %option noyywrap，Flex 示例无需链接 -lfl 或 -ll。

操作要点

（1）打开 Linux 终端、macOS 终端或 Windows 的 WSL 终端；Git Bash 本身不提供完整编译工具链

（2）确认 GCC 已安装，命令：gcc --version

（3）建立统一目录：mkdir -p ~/compiler_lab/project1 && cd ~/compiler_lab/project1

关键命令

```
# 在同一工作目录执行；macOS 的 gcc 可能实际为 Apple Clang。
$ gcc --version
$ flex --version
# Linux：gdb --version；macOS：lldb --version
$ cat > check.c << 'EOF'
#include <stdio.h>
int add(int a, int b) { return a + b; }
int main(void) {
    printf("add(2,3)=%d\n", add(2, 3));
    return 0;
}
EOF
$ gcc -Wall -Wextra -O0 -g check.c -o check
$ ./check
# 预期：add(2,3)=5
$ nm check | grep -E 'main|add'
# Linux：objdump -d check；ldd check
# macOS：otool -tv check；otool -L check
# Linux：gdb -batch -ex quit ./check
# macOS：lldb -b -o quit ./check
$ cat > demo.l << 'EOF'
%option noyywrap
%{
#include <stdio.h>
%}
%%
[0-9]+       { printf("INT: %s\n", yytext); }
[ \t\r\n]+  { /* skip whitespace */ }
.           { /* skip other characters */ }
%%
int main(void) { yylex(); return 0; }
EOF
$ flex demo.l
$ gcc -o demo lex.yy.c
$ printf '123 hello 456\n' | ./demo
# 预期：分两行输出 INT: 123 和 INT: 456
```

自检通过的标准

- check.c 编译成功，运行输出 add(2,3)=5；本例不要求出现警告
- nm 能找到 main/add（macOS 常显示 _main/_add）；地址不要求与示例相同
- objdump 或 otool 能显示本机架构的指令

- ldd 或 otool -L 能显示动态库依赖，具体库名因平台而异
- GDB 或 LLDB 能加载带调试信息的程序并正常退出
- Flex 生成的 demo 可运行并识别两个整数，不仅是生成或编译成功

提交要求：课前群内打卡“环境配置完成 + 截图”；报告第 1 章简要说明操作系统、架构及工具版本，不另交环境报告

任务 1.1 走近 GCC：从源代码到可执行文件

编译 四阶段入门

教学目标

- 认识 GCC 工具链的总体构成与各阶段分工
- 掌握 C 程序从源代码到可执行文件的最简流程
- 理解 .c / .i / .s / .o / 可执行文件 五种文件类型的本质
- 理解编译单元翻译与链接的分工：链接负责符号解析、重定位和程序组成；不以文件数作为唯一区分

GCC 是驱动程序。“四阶段”描述外部工具链；词法分析、语法分析、语义分析、IR、优化和代码生成属于编译器内部工作。-o 只指定输出名，不决定停止阶段。

操作要点

（1）创建源文件 hello.c：使用编辑器，或执行下方“步骤 1”的完整终端命令。

（2）编译：gcc hello.c -o hello，运行 ./hello

（3）用 file hello 验证可执行文件类型（Linux: ELF；macOS: Mach-O）

（4）故意制造语法错误（缺右括号等），观察 GCC 错误信息

（5）拆分多文件 add.c + main.c，理解链接错误

关键命令

命令说明：在 Bash 或 Zsh 终端中执行。hello.c 是文件名；> 将内容写入文件（会覆盖同名文件）；<< 'EOF' 开始多行输入，这种写法称为 heredoc。最后的 EOF 是结束标记，须单独成行且行首不留空格，不写入文件；引号使正文按原样写入。示例中的 $ 是终端提示符，不需输入。也可用编辑器创建同名文件，只保存 C 源代码。

```
# ———————————————— 步骤 1：写一个 hello.c ————————————————
$ cat > hello.c << 'EOF'
#include <stdio.h>
#define GREETING "Hello, Compiler!"
int add(int a, int b) { return a + b; }
int main(void) {
    int x = 2, y = 3;
    printf("%s %d + %d = %d\n", GREETING, x, y, add(x, y));
    return 0;
}
EOF
# ———————————————— 步骤 2：编译并运行 ————————————————
$ gcc hello.c –o hello
```

```
$ ./hello
Hello, Compiler! 2 + 3 = 5
# ———————————— 步骤 3：制造语法错误 ————————————
$ cat > bug.c << 'EOF'
#include <stdio.h>
int main(void) {
    printf("hello"  // 缺少右括号
    return 0;
}
EOF
$ cp bug.c bug_fixed.c
```

先用 gcc bug.c -o bug 查看诊断，再用编辑器修复 bug_fixed.c，保留 bug.c。记录错误原因、修改内容及修复后的输出。

```
$ gcc bug_fixed.c –o bug_fixed && ./bug_fixed
```

下面的诊断仅示意错误性质；错误行列和措辞以本机编译器为准。

```
$ gcc bug.c –o bug
bug.c: In function 'main':
bug.c:4:5: error: expected ')' before 'return'
    4 |     return 0;
      |     ^~~~~~
# ———————————— 步骤 4：多文件项目 ————————————
$ cat > add.c << 'EOF'
int add(int a, int b) { return a + b; }
EOF
$ cat > main.c << 'EOF'
#include <stdio.h>
extern int add(int, int);
int main(void) {
    printf("3 + 5 = %d\n", add(3, 5));
    return 0;
}
EOF
$ gcc main.c –o broken          # 期望：链接错误（undefined reference）
/usr/bin/ld: main.o: in function 'main':
main.c:(.text+0x15): undefined reference to 'add'
collect2: error: ld returned 1 exit status
$ gcc main.c add.c –o two_files   # 期望：编译链接成功
$ ./two_files
3 + 5 = 8
```

验收点

- ①Hello World 正确输出（Hello, Compiler! 2 + 3 = 5）
- ②能读懂 GCC 错误信息
- ③能正确编译多文件项目

提交要求：hello.c、bug.c、bug_fixed.c、add.c、main.c 源码

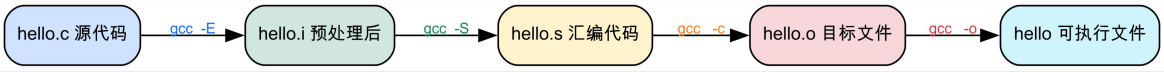


图 2　GCC 编译 四阶段流水线

任务 1.2 阶段产物观察

四阶段产物的检查方法

教学目标

- 深入理解 四阶段（预处理、编译、汇编、链接）的输入/输出
- 掌握 GCC 各阶段对应选项（-E / -S / -c）
- 通过对比源代码、汇编代码和可执行文件，理解各阶段的作用

四阶段工具链

表 4　工具链阶段与产物

阶段	工具	输入	输出	GCC 选项
预处理	cpp	.c + 头文件	.i（预处理 C 文本，可含行标记）	-E
编译	cc1	.i	.s（汇编代码）	-S
汇编	as	.s	.o（目标文件）	-c
链接	ld	目标文件与库	可执行文件	不加 -E/-S/-c；-o 仅指定名称

操作要点

（1）gcc -E hello.c -o hello.i，记录头文件展开后的行数；行数随头文件和平台变化

（2）gcc -S hello.i -o hello.s，查找 main:（macOS 通常为 _main:）标签

（3）gcc -c hello.s -o hello.o，用 file / nm / objdump 观察

（4）gcc hello.o -o hello_from_o，观察链接中的符号解析与目标文件组合

（5）gcc -### hello.c 查看计划调用的内部命令；该选项本身不执行编译

关键命令

```
# 以下以 Linux 上的 GCC 为基准；输出细节以本机为准。
$ gcc –E hello.c –o hello.i
$ wc –l hello.i
$ head –10 hello.i
$ grep –c '^#' hello.i
# 以 # 开头的行可能是文件和行号标记，并非尚未展开的 #include。
$ gcc –E –P hello.c –o hello_no_markers.i
$ head –10 hello_no_markers.i
# –P 抑制预处理输出中的行标记。
$ gcc –S hello.i –o hello.s
$ grep –A 3 –E '^_?main:' hello.s
# Linux 通常为 main:；macOS 通常为 _main:。
$ gcc –c hello.s –o hello.o
$ file hello.o
$ nm hello.o
# Linux：观察段与重定位。
$ objdump –h hello.o
$ objdump –r hello.o
# macOS 改用：otool –l hello.o；otool –rv hello.o。
$ gcc hello.o –o hello_from_o
$ ./hello_from_o
$ gcc –c add.c –o add.o
```

```
$ gcc –c main.c –o main.o
$ gcc add.o –o no_main
# 预期链接失败：缺少程序入口所需的 main。
$ gcc main.o add.o –o two_files
$ ./two_files
# Linux：ldd hello_from_o；macOS：otool –L hello_from_o。
$ gcc –### hello.c –o hello_trace 2> trace.txt
$ cat trace.txt
# –### 只打印计划调用的命令，不执行，不生成 hello_trace。
# GCC 通常调用 cc1、as 和 collect2/ld；临时 .s/.o 名称因版本而异。
# Clang 的前端和集成汇编器组织不同，不能硬套 GCC 内部命令。
```

验收点

- ①观察源文件 .c、阶段产物 .i/.s/.o 和可执行文件；这是五种文件，不是五个编译阶段
- ②说明 -E/-S/-c 的停止位置，并解释 -### 只打印命令的作用

提交要求：报告含 四阶段命令、产物截图、关键观察

观察要点：.i 中 GREETING 已展开；.s 中有函数标签；.o 中包含符号与重定位记录；链接后可以运行。as 接收汇编 .s，不接收预处理 C 文本 .i。

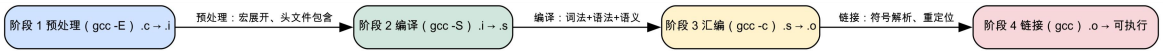


图 3　四阶段工具链总览

任务 1.3 GCC 编译选项实验

性能与诊断的取舍

教学目标

- 熟悉 GCC 常用选项（优化、警告、调试、预处理宏）
- 比较不同编译选项对文件大小、汇编代码和运行时间的影响
- 能够根据调试和运行需求选择编译选项

实验内容

表 5　实验内容

编号	名称	说明
实验 1	优化等级对比（必做）	对比 -O0 / -O1 / -O2 / -O3 的文件大小、汇编行数、执行时间
实验 2	警告与错误（必做）	使用 -Wall -Wextra -Werror 至少识别 5 种警告类型
实验 3	调试信息（必做）	对比 -g 与不带 -g 的 gdb 体验差异
实验 4	预处理宏（必做）	用 -DDEBUG 开关条件编译

操作要点

实验 1：构造循环密集型程序 loop.c（见代码块）

实验 2：故意写有未初始化变量、赋值当比较的代码，用 -Wall -Wextra -Werror 编译

```
实验 3：gcc –O0 –g test.c –o test_dbg && gdb ./test_dbg 看源码调试
实验 4：gcc –DDEBUG test.c –o test_debug 开关条件编译
```

先用公式 $\sum_{i=0}^{N-1}(2i+1)=N^2$验证输出。使用 long long 避免 long 在部分平台仅为 32 位。优化可能直接化简该循环，因此本例适合观察语义保持与产物变化，不能代表一般程序的性能。

关键命令

```
# 实验 1：优化等级对比
$ cat > loop.c << 'EOF'
#include <stdio.h>
#include <time.h>
#define N 10000000LL
int main(void) {
    long long sum = 0;
    clock_t start = clock();
    for (long long i = 0; i < N; i++)
        sum += i * 2 + 1;
    clock_t end = clock();
    if (start == (clock_t)-1 || end == (clock_t)-1) {
        fputs("CPU time unavailable\n", stderr);
        return 1;
    }
    printf("sum = %lld\n", sum);
    printf("loop CPU time = %.6f s\n",
           (double)(end - start) / CLOCKS_PER_SEC);
    return 0;
}
EOF
$ gcc -O0 loop.c -o loop_O0
$ gcc -O1 loop.c -o loop_O1
$ gcc -O2 loop.c -o loop_O2
$ gcc -O3 loop.c -o loop_O3
$ gcc -O0 -S loop.c -o loop_O0.s
$ gcc -O1 -S loop.c -o loop_O1.s
$ gcc -O2 -S loop.c -o loop_O2.s
$ gcc -O3 -S loop.c -o loop_O3.s
$ wc -c loop_O0 loop_O1 loop_O2 loop_O3
$ wc -l loop_O0.s loop_O1.s loop_O2.s loop_O3.s
$ time ./loop_O0
$ time ./loop_O3
# 各版本 sum 均为 100000000000000；CPU 时间以本机实测为准
# 实验 2：警告与错误
$ cat > warn.c << 'EOF'
#include <stdio.h>
int missing_return(void) { }
int main(void) {
    int unused = 0;
    int x;
    int y = 0;
    if (y = 5) printf("y=%d\n", y);
    printf("%d\n", "wrong type");
    return x;
}
EOF
$ gcc -O1 -Wall -Wextra -c warn.c -o warn.o
# 观察：缺少返回值、未使用变量、赋值作为条件、
# 格式与实参类型不匹配、读取未初始化变量。
# 本例有未定义行为，只用于诊断，禁止链接运行。
$ gcc -O1 -Wall -Wextra -Werror -c warn.c -o warn.o
# -Werror 将诊断中的警告提升为错误；编译预期失败。
```

```
# 实验 3/4：调试信息与条件编译
$ cat > test.c << 'EOF'
#include <stdio.h>
int main(void) {
    int x = 40;
    x += 2;
#ifdef DEBUG
    printf("[DEBUG] x = %d\n", x);
#endif
    printf("answer = %d\n", x);
    return 0;
}
EOF
$ gcc -O0 test.c -o test_no_gdb
$ gcc -O0 -g test.c -o test_with_gdb
$ gdb ./test_with_gdb
# 在 GDB 中依次输入：break main、run、next、print x、quit。
# 再用 gdb ./test_no_gdb 对照源代码行和局部变量信息。
# 无 -g 时仍可能看到符号名或反汇编，不等于完全无法调试。
# macOS 使用 LLDB：lldb ./test_with_gdb
# 输入：breakpoint set --name main、run、next、frame variable、quit。
$ gcc -O0 test.c -o test_off
$ ./test_off
# 预期：answer = 42
$ gcc -O0 -DDEBUG test.c -o test_on
$ ./test_on
# 预期：先输出 [DEBUG] x = 42，再输出 answer = 42
```

记录实际编译器版本、CPU/架构和编译选项。程序用 C 标准库 clock() 记录循环区间消耗的 CPU 时间，以秒输出 loop CPU time；不包含后续 printf 的时间，也不是墙钟时间。time ./loop_O0 等终端命令可另行观察整个进程的 real/user/sys。每个优化版本至少运行 5 次，分别记录循环 CPU 时间的中位数；0 或接近计时分辨率的结果应标注“无法可靠区分”。循环可能被编译期化简，不要为了计时擅自加入 volatile 改变本实验的优化条件。

警告诊断随编译器、选项和版本变化。记录实际诊断类别并修复代码；不要照抄行号和文本。-g 产生调试信息，-O0 有助于源码单步对应；DEBUG 是本程序定义的条件编译宏，与调试信息选项 -g 不等价。

验收点

- ①4 种优化等级对比（含文件大小、汇编行数、执行时间）
- ②至少 5 种警告类型
- ③-g vs 不带 -g 的 gdb 体验差异

提交要求：4 个实验的实际记录、优化对比表及结论依据；截图可汇总

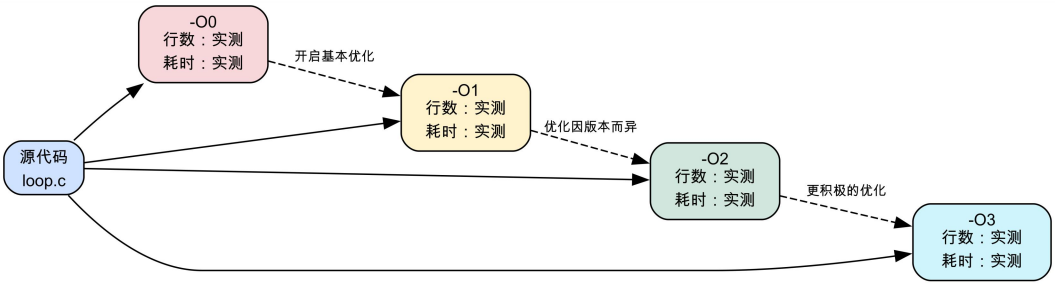


图 4　优化等级对比示意

任务 1.4 Lex 工具入门

编写与测试简单的词法分析器

教学目标

- 了解 Lex/Flex 工具的 三段式 结构与工作流
- 亲手编写并运行最简单的 .l 文件
- 了解正规式如何通过自动机构造用于词法分析
- 为项目 2 任务 2.1 的 Thompson 构造学习做准备，认识工具背后的自动机算法

三个实验依次练习工具使用、多类规则编写和简单语言词法规则的组织。实验 1 提供完整示例；实验 2 和实验 3 的识别规则由学生完成。

实验内容

表 6　实验内容

编号	名称	说明
实验 1	最简整数识别器	运行完整示例，认识 .l 文件结构及 Flex 的生成、编译和运行流程
实验 2	多类词法单元识别——以数字字面量为例	编写十进制、八进制、十六进制整数和简单浮点数规则，输出类别与匹配文本
实验 3	小型 C 风格词法识别器	组织关键字、标识符和整数规则，连续识别输入文本，观察最长匹配与等长规则优先级

操作要点

实验 1：运行 demo.l，完成 Flex 生成 C 文件、编译和运行的全过程，说明各文件之间的关系。

实验 2：在 numbers.l 中编写多条数字字面量规则，按类别输出匹配文本，检查边界输入的切分结果。

实验 3：在 mini_c.l 中组织关键字、标识符和整数规则，连续识别输入文本，并通过调整规则顺序观察匹配选择。

关键命令

实验 1：最简整数识别器

```
$ cat > demo.l << 'EOF'
%option noyywrap
%{
#include <stdio.h>
%}
%%
[0-9]+       { printf("INT: %s\n", yytext); }
[ \t\r\n]+  { /* skip whitespace */ }
.           { /* skip other characters */ }
%%
int main(void) { yylex(); return 0; }
EOF
$ flex demo.l
$ gcc -o demo lex.yy.c
$ printf '123 hello 456\n' | ./demo
# 预期：分两行输出 INT: 123 和 INT: 456
```

实验 2 多类词法单元识别——以数字字面量为例

在实验 1 的基础上，将单条整数识别规则扩展为多条词法规则。本实验选择不同形式的数字字面量作为识别对象，练习为不同输入模式编写正规式，并通过规则动作输出识别类别和匹配文本。

将学生配套中的 numbers.l 复制到实验目录。框架可以生成和编译，但目前只跳过空白，并以 UNHANDLED 提示未处理字符。请在空白规则之前补充四类数字字面量规则，保留末尾的未处理字符提示。

识别要求：十进制整数为 0 或以非零数字开头的数字串；八进制整数以 0 开头，后接至少一位 0—7 数字；十六进制整数以 0x 或 0X 开头，后接至少一位十六进制数字；简单浮点数的小数点两侧均有数字，不含指数部分。分别使用 DEC、OCT、HEX、FLOAT 标记输出类别，并输出匹配文本。

为便于核对匹配情况，本实验将不同进制的整数分别标记为 DEC、OCT、HEX。它们属于整数的不同表示形式，实际语言的词法分析器也可以统一识别为整数常量，在属性中保存原文或数值。

本实验关注输入文本如何被规则识别和切分，不要求计算表达式，也不要求完整诊断非法数字。负号不属于本实验的数字规则；若增加负号识别，应将其作为独立运算符。

```
$ flex numbers.l
$ gcc -o numbers lex.yy.c
$ printf '123 0x1F 075 3.14 0\n' | ./numbers
```

上述五项应依次输出 DEC、HEX、OCT、FLOAT、DEC 及相应匹配文本。另选至少两组边界输入（如 08、0x、.5、-12），先预测切分和分类结果，再运行核对并解释原因。对于不符合本实验数字格式的输入，应记录实际切分及未处理字符，不能把若干局部匹配当作整个数字合法。

实验 3 小型 C 风格词法识别器

本实验将识别对象从数字字面量扩展到简单的 C 风格输入文本，练习组织多条词法规则，并按输入顺序连续输出识别结果。

在 mini_c.l 框架中组织三类规则：int、if、while 关键字；标识符；十进制整数。标识符以英文字母或下划线开头，后接零个或多个英文字母、数字或下划线。整数规则采用 [0-9]+。分别使用 KW、ID、INT 标记输出类别，并按输入顺序输出匹配文本；跳过空白，保留对未处理字符的提示。

```
$ flex mini_c.l
$ gcc –o mini_c lex.yy.c
$ printf 'int integer if ifx while while2 _x 42\n' | ./mini_c
```

先预测各词法单元的类别，再运行核对。随后交换关键字规则与标识符规则的顺序，比较 int、integer、ifx 和 while2 的结果，记录修改前后的输出，并说明原因。

Flex 优先选择匹配文本最长的规则；长度相同时选择写在前面的规则。int 同时满足关键字和标识符规则，匹配长度相同，因此由规则顺序决定类别。integer 应整体识别为标识符，不能切分为关键字 int 和其余字符，因为标识符规则匹配的文本更长。ifx、while2 也应整体识别为标识符。

本实验只实现小型词法识别器，不要求覆盖 C 语言的全部词法规则，也不涉及语法分析。有余力的同学可增加少量运算符、分隔符或注释规则，用一段简单程序观察连续识别结果；这些内容作为扩展练习。

验收点

- 实验 1：能够生成、编译和运行示例，提交源码与运行记录，说明 .l 文件、生成的 C 文件和可执行文件之间的关系。
- 实验 2：完成四类数字字面量规则，提交类别与匹配文本，以及至少两组边界输入的预测、实际结果和解释。
- 实验 3：完成关键字、标识符和整数规则，提交规则顺序调整前后的结果，解释最长匹配与等长规则优先级，并给出一个自测反例。

共同理解要求：正规式用于描述词法规则，Flex 借助自动机完成识别。能够结合本实验说明这一关系即可，具体自动机构造算法在项目 2 中学习。

提交要求：demo.l、numbers.l 和 mini_c.l 源码；三个实验的预期与实际输出；边界输入记录；规则修改前后的对比及原因解释。

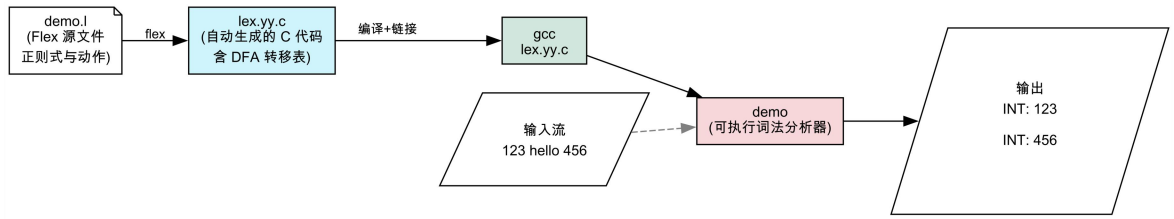


图 5　Flex 工具链

任务 1.5 国产编译器调研

科技报国，从基础软件做起

教学目标

- 了解国产编译器的发展现状与典型代表

- 体会编译器作为基础软件对国家信息安全的战略意义
- 树立 科技报国 的使命感

选题清单（6 选 1 或自选）

表 7　调研选题

选题	代表	主页
A	华为方舟编译器（Ark Compiler）	gitee.com/openarkcompiler/openarkcompiler
B	华为毕昇编译器（BiSheng）	gitee.com/openeuler/bishengcompiler
C	OpenHarmony 编译器体系	gitee.com/openharmony
D	龙芯 LoongArch 工具链	gitee.com/loongson
E	阿里云 / 字节跳动自研编译器	（内源项目）
F	自选	经教师确认

报告模板（800 字以上）

- 调研对象：所选国产编译器名称 + 一句话定位
- 关键发现：支持语言、目标架构、工具链组成和一项可核查的技术特征
- 我的思考：结合本项目，说明适用场景、局限或一次可复现体验
- 参考文献（≥ 2 条）：官方仓库/白皮书 + 博客/论文

提交要求：800 字以上调研报告及至少 2 条来源（含官方资料），随项目报告提交

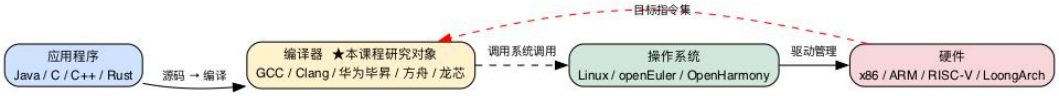


图 6　基础软件栈：编译器的位置

实验报告写作说明

1. 综合报告章节结构

表 8　实验报告结构

章节	内容
第 1 章	环境配置：课前打卡，报告简要说明平台、架构和工具版本
第 2 章	走近 GCC（任务 1.1）
第 3 章	阶段产物观察（任务 1.2）
第 4 章	编译选项实验（任务 1.3）
第 5 章	Lex 工具入门（任务 1.4）
第 6 章	思政融合：国产编译器调研（任务 1.5）
第 7 章	综合反思（最有趣的环节 / 最大收获 / 还存在的疑问）

2. 提交材料清单

表 9　提交材料清单

任务	提交材料	位置
1.0	环境自检截图及平台、架构、工具版本记录	班级群打卡；报告第 1 章
1.1	hello.c / bug.c / add.c / main.c / 编译运行截图	报告第 2 章
1.2	.i / .s / .o 产物 + nm/objdump/ldd 输出	报告第 3 章
1.3	优化等级对比表 + 5 种警告截图	报告第 4 章
1.4	demo.l 运行记录、完成的两个练习及测试预期与实际输出	报告第 5 章
1.5	800 字以上调研报告 + 参考文献	报告第 6 章

3. 提交时间

项目 1 上机结束后 1 周内

4. 验收方式

提交源文件及实际记录，并完成简短演示：解释一条阶段命令，定位并修复一个编译或链接错误，对一组新的 Flex 输入先预测再运行。完成示例运行是基础要求，评价还依据规则正确性、测试覆盖和观察解释，不以截图数量或修改行数评分。

5. 参考资料

GCC 阶段选项：https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html

GCC 行标记：https://gcc.gnu.org/onlinedocs/cpp/Preprocessor-Output.html

Flex 匹配规则：https://westes.github.io/flex/manual/Matching.html

AI 辅助编写说明

本次课程材料使用了多个 AI 辅助工具，协助开展实验任务设计、实验指南编写、示例整理以及文字和排版润色。材料由编写者统筹取舍，课程组负责审核与定稿。